

PARALLEL COMPUTING — CLASS 0 DIAGNOSTIC

LAB work

230103113 Dinmukhamed Altynbek

Task 1:

$x = 5, y = 2$

$x = 7, y = 2$

$x = 7, y = 14$

$x = 7, y = 14$

Final output: 7 14

TASK 2 - Find Bugs

The program calculates the last element of the array, which is 50.

It was probably intended to calculate the sum of all elements.

Fix:

`total += numbers[i]`

The result is 150.

TASK 3 - Nested loops

(a) `print()` executes $10 \times 10 = 100$ times.

(b) If both loops run from 0 to 999:

$1000 \times 1000 = 1,000,000$ times.

(c) If the outer loop runs 10 times and the inner loop runs 1,000,000 times:

$10 \times 1,000,000 = 10,000,000$ times.

TASK 4 - Which is faster?

Algorithm A examines each element once, so it does about n operations: $O(n)$.

Algorithm B compares every element with every element, so it does about n^2 operations: $O(n^2)$.

Algorithm B grows much faster.

For $n = 1,000,000$:

$A \approx 1,000,000$ operations.

$B \approx 1,000,000,000,000$ operations.

TASK 5

Fastest-growing to slowest-growing:

2^n
 n^2
 $n \log_2 n$
 n
 $\log_2 n$
1

2^n and n^2 become particularly problematic for very large n .

TASK 6 - searching

1) If the list is not known to be sorted, use linear search.

Check each element:

$3 \rightarrow 8 \rightarrow 12 \rightarrow 17 \rightarrow 24 \rightarrow 31 \rightarrow 45$

45 is found after 7 checks.

2) If the list is sorted, use binary search.

Check the middle element and eliminate half of the remaining list at each step.

Binary search is faster because it reduces the search space by half after each comparison.

TASK 7 Large Dataset

Use a single-pass algorithm.

Keep one variable called max.

Examine each number once and update max if the current number is larger.

Each number needs to be examined once.

We do not need to store all 500 million numbers.

We can process the data in chunks.

Memory usage can be very small because we only need the current number and the current maximum.

TASK 8 - counting

Use a dictionary/hash map.

For each element:

- if the element exists in the dictionary, increase its count;
- otherwise, create it with count 1.

Result:

$A \rightarrow 4$

B → 3
C → 2
D → 1

The data structure is a dictionary/hash map.

TASK 9 - Matrix operation

A = [[1,2],
 [3,4]]

B = [[5,6],
 [7,8]]

Additional

A + B = [[6,8],
 [10,12]]

Multiplication

A × B = [[19,22],
 [43,50]]

Individual multiplication operations:

$2 \times 2 \times 2 = 8$

So, 8 multiplication operations.

TASK 10 - CPU vs Memory

Possible reasons:

1. The program is memory-bound and spends time waiting for data from memory.
2. The array is too large for the CPU cache.
3. Memory bandwidth is the bottleneck.

TASK 11 What Happens When You Run a Program?

When I type ./program, the operating system starts the program.

The executable is loaded into memory.

The operating system creates a process and allocates resources.

The CPU begins executing the program instructions.

The program reads input, performs calculations and may access memory or other resources.

Eventually the program produces output, which is sent to the terminal.

When the program finishes, the operating system releases its resources.

Task 12 — Four Programs

A: CPU-bound — it uses lots of CPU.

B: Memory-bound — it uses lots of memory.

C: I/O-bound — it spends time reading a large file.

D: I/O-bound — it mostly waits for user input.

Yes, all four can run at the same time if the operating system schedules them.

CPU time is determined by the operating system scheduler and available CPU cores.

With one CPU core, the programs cannot execute CPU instructions simultaneously. The operating system switches between them.

Task 13 — The Slow Program

Not necessarily.

If the program is completely CPU-bound and the new computer is twice as fast, it could take approximately 5 hours.

However, if the program spends time waiting for memory, disk, network, or other resources, the total time may not be reduced by half. Therefore, it depends on the bottleneck.

Task 14 — Make It Faster

Possible ways:

- Improve the algorithm.
- Reduce the number of operations.
- Use a more efficient data structure.
- Avoid unnecessary calculations.
- Reduce memory accesses.
- Use CPU cache more effectively.
- Use faster I/O.
- Process data in chunks.
- Use parallel processing.
- Use multiple CPU cores.
- Use vectorization/SIMD where appropriate.
- Reduce unnecessary copying.
- Optimize the bottleneck.
- Use compiler optimizations.

Task 15 — Bottleneck Identification

A = 2s

B = 2s

C = 50s

D = 3s

E = 2s

Total = 59s

A, B, D и E становятся в 10 раз быстрее:

A = 0.2s

B = 0.2s

C = 50s

D = 0.3s

E = 0.2s

Новое время:

$0.2 + 0.2 + 50 + 0.3 + 0.2 = 50.9s$

Ускорение всей программы:

$59 / 50.9 \approx 1.16\times$

The entire program becomes only about 1.16 times faster.

The main bottleneck is C because it takes 50 seconds.

Optimization effort should focus on C.

Task 16 — 10 Million Student Marks

Use one pass through the data to calculate:

- minimum
- maximum
- sum/average
- number of scores ≥ 90
- number of scores < 40

For the median, because marks are between 0 and 100, use a frequency array with 101 positions.

For example:

count[0] = number of students with mark 0

count[1] = number of students with mark 1

...

count[100] = number of students with mark 100.

Data structures:

- variables for min, max, sum and counters
- an array of 101 integers for the frequencies

The 10 million marks need to be examined once: $O(n)$.

Memory requirement is small because the frequency array has only 101 elements. We do not need to store all 10 million marks.

The likely bottleneck is reading/processing the 10 million records.

To make it faster, process the data in chunks and use parallel processing if appropriate. Multiple processors/cores can process different chunks at the same time.

The final results can then be combined.